

AKILI

Automation Language

AAL v1.0 — Formal Language Specification

Document	AAL-SPEC-001
Version	1.0.0
Status	DRAFT FOR CERTIFICATION
Author	Eric Fanane, CrossTech Path
Date	May 2026
Classification	Confidential & Proprietary
Copyright	© 2026 CrossTech Path. All rights reserved.

CrossTech Path | crosstechpath.com | Registered in France

1. Abstract

The AKILI Automation Language (AAL) is a domain-specific language (DSL) designed for industrial automation control in resource-constrained environments. AAL enables operators, technicians, and engineers to define automation logic using structured natural language constructs without requiring knowledge of traditional programmable logic controller (PLC) programming languages such as Ladder Diagram (LD) or Structured Text (ST) as defined in IEC 61131-3.

AAL is optimized for deployment on embedded Linux systems (Raspberry Pi class hardware) running the AKILI IoT Platform developed by CrossTech Path. It targets industrial and agricultural automation use cases in Africa and developing regions where access to trained PLC engineers is limited.

This specification defines the syntax, semantics, execution model, and standard library of AAL v1.0. It is intended to serve as the basis for language certification, implementation validation, and third-party integrations.

2. Scope and Objectives

2.1 Design Philosophy

AAL is founded on four core principles:

1. **Human Readability** — Rules must be understandable by a trained technician without a programming background. A nurse, farmer, or factory operator should be able to read an AAL rule and understand what it does.
2. **Determinism** — Given the same inputs and state, an AAL rule must always produce the same output. No ambiguity in execution.
3. **Safety by Default** — When the system is uncertain (sensor lost, rule conflict, hardware fault), it defaults to the safe state: all outputs OFF.
4. **Minimalism** — The language has the minimum constructs necessary to cover 95% of industrial automation use cases. Complexity is a defect.

2.2 Target Users

AAL is designed for three user levels:

User Level	Background	Expected Capability
Operator	No programming experience	Read and understand existing rules
Technician	Basic technical training	Create and modify simple rules
Engineer	Engineering or IT background	Design complex sequences and math expressions

2.3 Scope of Coverage

AAL covers the following automation domains:

- Cold chain monitoring and control
- Pharmaceutical storage (WHO GDP compliant)
- Agricultural automation (greenhouse, irrigation, livestock)
- Refrigerated transport
- Industrial process control
- Building automation
- Water treatment and distribution
- Power management

3. Language Overview

3.1 Paradigm

AAL is an event-driven, declarative language. Programs are expressed as a collection of independent rules and sequences. Each rule specifies:

5. A set of conditions that must be true for the rule to fire (trigger logic)
6. A set of actions to execute when the rule fires
7. Optionally, conditions under which the rule resets
8. Optionally, timing constraints (activation delay, minimum ON time, maximum ON time, cooldown)

AAL does not use explicit loops, variable declarations, or subroutines. State is maintained implicitly by the runtime through sensor readings, virtual variables, and event counters.

3.2 Execution Model

The AAL runtime evaluates all active rules every poll cycle. The poll cycle period is configurable (default: 30 seconds). Evaluation order follows rule priority:

Priority	Level	Behaviour
1 — CRITICAL	Highest	Evaluated first. Can override PID controllers and lower-priority rules. Used for safety and emergency logic.
2 — HIGH	High	Evaluated second. Can override PID setpoints. Used for operational priority logic.
3 — NORMAL	Standard	Default priority. Runs alongside PID controllers without override.
4 — LOW	Background	Evaluated last. Used for logging, monitoring, and non-critical automation.

Safe State Guarantee

If the AKILI runtime terminates unexpectedly, all physical relay outputs are set to OFF (de-energized) before the process exits. This safe state is also applied at startup before any rules are

evaluated. No rule can disable the safe state mechanism.

4. Syntax Specification

4.1 Rule Structure

An AAL rule has the following formal structure:

```
RULE <name>
  PRIORITY <CRITICAL | HIGH | NORMAL | LOW>
  ENABLED <TRUE | FALSE>

  -- Trigger conditions (required)
  IF <condition>
  [AND <condition>]
  [OR <condition>]
  [AND NOT <condition>]

  -- Timing (optional)
  FOR <duration>           -- Debounce: condition must hold this long
  MIN_ON <duration>        -- Relay stays ON at least this long
  MAX_ON <duration>        -- Safety: auto-reset after this duration
  COOLDOWN <duration>     -- Wait after reset before re-triggering

  -- Actions on fire (required)
  THEN <action>
  [AND <action>]

  -- Reset conditions (optional)
  RESET WHEN <condition>
  [AND <condition>]

  -- Actions on reset (optional)
  ON RESET <action>
  [AND <action>]

  NOTES "<text>"
END RULE
```

4.2 Duration Format

Durations are expressed as a positive real number followed by a unit:

Unit	Symbol	Example
Seconds	s or sec	30s, 5sec
Minutes	m or min	5m, 10min
Hours	h or hr	2h, 1hr

4.3 Condition Types

4.3.1 Sensor Comparison Conditions

```
IF <sensor_id> <operator> <value>

Operators: > | < | >= | <= | = | !=
sensor_id: alphanumeric identifier as defined in the module configuration
value: numeric literal
```

Examples:

```
IF cargo_temp_front > 8.0
IF mains_power = 0
IF soil_moisture < 40.0
```

4.3.2 Rate of Change Conditions

```
IF <sensor_id> RISING > <value_per_minute>
IF <sensor_id> FALLING > <value_per_minute>
```

Evaluates the rate of change over the last 60 seconds. Units are sensor units per minute.

```
IF cargo_temp_front RISING > 2.0 -- Temperature rising faster than
2°C/min
```

4.3.3 Sensor Loss Condition

```
IF <sensor_id> NO_DATA > <duration>
```

Triggers when a sensor has not reported a value for longer than the specified duration. Used to detect sensor faults or communication failures.

```
IF cold_storage_1 NO_DATA > 5min
```

4.3.4 Relay State Conditions

```
IF RELAY <channel> IS ON
IF RELAY <channel> IS OFF
```

```
IF RELAY 2 IS ON -- True when relay channel 2 is energized
```

4.3.5 Time Conditions

```
IF TIME BETWEEN <HH:MM> AND <HH:MM>
```

```
IF TIME AFTER <HH:MM>
IF TIME BEFORE <HH:MM>
IF DAY IN <Mon | Tue | Wed | Thu | Fri | Sat | Sun>
```

```
IF TIME BETWEEN 22:00 AND 06:00 -- Crosses midnight correctly
IF DAY IN Mon, Wed, Fri
```

4.3.6 Variable Conditions

```
IF VARIABLE <name> = <value>
```

```
IF VARIABLE generator_running = 1
```

4.3.7 Counter Conditions

```
IF COUNTER <name> >= <count> IN <duration>
```

```
IF COUNTER door_open_events >= 3 IN 1hr
```

4.3.8 Math Expression Conditions

```
IF EXPR "<expression>" <operator> <value>
```

The expression may reference any sensor ID as a variable. Supported functions: abs, round, min, max, sqrt, floor, ceil, pow, log, sin, cos, pi, e.

```
IF EXPR "(temp_zone1 + temp_zone2) / 2" > 28.0
IF EXPR "abs(cold_1 - cold_2)" > 2.0
```

4.3.9 TON Timer Condition

```
IF SENSOR <sensor_id> <operator> <threshold> FOR_TIMER <duration>
```

Equivalent to IEC 61131-3 TON (Timer ON delay). The condition becomes true only after the sensor has continuously satisfied the comparison for the specified duration. Resets immediately when the sensor returns to normal.

```
IF SENSOR ammonia_level > 20.0 FOR_TIMER 30s
```

4.3.10 Unconditional Condition

```
IF ALWAYS
```

The rule fires on every poll cycle. Used for periodic actions such as data logging or heartbeat signals.

4.4 Logical Operators

Multiple conditions are combined using logical operators. The first condition has no operator (implicit). Subsequent conditions use:

Operator	Meaning	Behaviour
AND	Logical AND	Both conditions must be true
OR	Logical OR	Either condition must be true
AND NOT	Logical AND NOT	First true AND second false
OR NOT	Logical OR NOT	First true OR second false

Operators are evaluated left to right with no operator precedence. Parenthetical grouping is not supported in v1.0.

5. Action Types

5.1 Relay Actions

Action Syntax	Description
RELAY <ch> ON	Energize relay channel ch (1-8)
RELAY <ch> OFF	De-energize relay channel ch
RELAY <ch> PULSE <duration>	Energize relay for duration then de-energize
RELAY <ch> TOGGLE	Invert current relay state

5.2 PID Override Actions

Action Syntax	Description
PID <sensor_id> SETPOINT <value>	Override the PID setpoint for the specified sensor
PID <sensor_id> RELEASE	Release PID override, return to configured setpoint

5.3 Notification Actions

Action Syntax	Description
SMS "<message>"	Send SMS to configured alert phone number
EMAIL "<message>"	Send email to configured alert address
LOG "<message>"	Write entry to the audit log

5.4 Variable Actions

Action Syntax	Description
SET <name> = <value>	Set a virtual variable to a numeric value
SET <name> = EXPR "<expression>"	Calculate expression and store result
INCREMENT COUNTER <name>	Increment a named event counter

5.5 Control Flow Actions

Action Syntax	Description
TRIGGER RULE <rule_id>	Force immediate evaluation of another rule
START SEQUENCE <sequence_id>	Start a named sequence
STOP SEQUENCE <sequence_id>	Stop a running sequence

6. Sequence Specification

6.1 Overview

A Sequence implements the Sequential Function Chart (SFC) concept from IEC 61131-3 using AAL syntax. Sequences are used for multi-step processes where steps must execute in order.

6.2 Sequence Structure

```
SEQUENCE <name>
  MODE <SINGLE | CYCLIC | HOLD>
  ENABLED <TRUE | FALSE>

  -- Optional: conditions to auto-start
  START WHEN <condition>

  -- Steps (one or more)
  STEP 1 "<name>"
    TIMEOUT <duration>
    ON ENTER <action> [AND <action>]
    WHILE ACTIVE <action> [AND <action>]
    ON EXIT <action> [AND <action>]
    ADVANCE WHEN <condition> GOTO STEP <n>
  END STEP

  STEP 2 "<name>"
```



```
...  
END STEP  
  
END SEQUENCE
```

6.3 Sequence Modes

Mode	Behaviour
SINGLE	Sequence runs once from Step 1 to last step, then stops. Status becomes COMPLETE.
CYCLIC	After the last step completes, the sequence restarts from Step 1 automatically.
HOLD	Sequence pauses at each step until manually advanced or ADVANCE WHEN condition is met.

6.4 Step Lifecycle

Each step has three phases executed in order:

9. ON ENTER — Executed once when the step becomes active. Used to set initial state (turn relay ON, send SMS, set variable).
10. WHILE ACTIVE — Executed every poll cycle while the step is active. Used for monitoring or continuous actions.
11. ON EXIT — Executed once when the step transitions to the next step. Used for cleanup.

6.5 Step Advancement

A step advances to the next step when:

12. An ADVANCE WHEN condition is met (evaluated every poll cycle)
13. The step TIMEOUT expires (if configured)
14. An operator manually advances the sequence

If no ADVANCE WHEN condition and no TIMEOUT are defined, the step holds indefinitely until manual advance.

7. Complete Examples

7.1 Generator Auto-Start (Logic Rule)

Start a generator relay when mains power fails. Stop it when power is restored. Include cooldown to prevent rapid cycling.

```
RULE "Generator Auto-Start"  
  PRIORITY CRITICAL  
  ENABLED TRUE
```

```
IF      mains_power = 0
FOR      5s

THEN     RELAY 4 ON
AND      SMS "Mains power failed – generator started"
AND      LOG "Generator started due to mains failure"

RESET WHEN mains_power = 1
ON RESET  RELAY 4 OFF
AND      SMS "Mains power restored – generator stopped"

MIN_ON    30s
COOLDOWN  5min
NOTES     "Clinic Lagos – backup power"
END RULE
```

7.2 Cold Chain Temperature Alarm (Logic Rule)

Trigger an alarm when vaccine cold storage exceeds WHO limit. Only fire if the door is also closed (ruling out door-open events).

```
RULE "Vaccine Cold Chain Alarm"
  PRIORITY CRITICAL
  ENABLED TRUE

  IF      cold_storage_1 > 8.0
  AND NOT door_sensor    = 1
  FOR      2min

  THEN     RELAY 2 ON
  AND      SMS "CRITICAL: Vaccine storage at risk – temp above 8°C"
  AND      EMAIL "Temperature excursion detected in cold storage"

  RESET WHEN cold_storage_1 < 6.0
  ON RESET  RELAY 2 OFF
  AND      LOG "Cold chain excursion resolved"

  MAX_ON    2hr
  NOTES     "WHO PQS E006 compliance rule"
END RULE
```

7.3 Night Greenhouse Heating (Time-Based Rule)

Activate heating during cold nights. Use a math expression to detect temperature drop across two zones.

```
RULE "Night Greenhouse Heating"
```

```
PRIORITY    NORMAL
ENABLED     TRUE

IF    TIME   BETWEEN 20:00 AND 06:00
AND   EXPR   "(gh_temp1 + gh_temp2) / 2" < 18.0

THEN  RELAY 1 ON
AND   LOG    "Night heating activated"

RESET WHEN  EXPR   "(gh_temp1 + gh_temp2) / 2" > 22.0
ON RESET   RELAY 1 OFF

MIN_ON     10min
NOTES      "Prevent cold stress on crops between 20:00 and 06:00"
END RULE
```

7.4 Irrigation Sequence (Multi-Step)

A 3-zone drip irrigation sequence. Each zone irrigates for 20 minutes in CYCLIC mode, repeating every morning.

```
SEQUENCE    "Morning Irrigation"
MODE         CYCLIC
ENABLED      TRUE
START WHEN   TIME   AFTER 06:00
AND          DAY    IN  Mon, Tue, Wed, Thu, Fri, Sat, Sun

STEP 1       "Irrigate Zone A"
  TIMEOUT    20min
  ON ENTER   RELAY 1 ON
  AND        LOG    "Irrigation Zone A started"
  ON EXIT    RELAY 1 OFF
  ADVANCE WHEN  soil_moisture_a >= 75.0
END STEP

STEP 2       "Irrigate Zone B"
  TIMEOUT    20min
  ON ENTER   RELAY 2 ON
  ON EXIT    RELAY 2 OFF
  ADVANCE WHEN  soil_moisture_b >= 75.0
END STEP

STEP 3       "Irrigate Zone C"
  TIMEOUT    20min
  ON ENTER   RELAY 3 ON
  ON EXIT    RELAY 3 OFF
  ADVANCE WHEN  soil_moisture_c >= 75.0
  AND        LOG    "All zones irrigated"
```

```
END STEP  
  
END SEQUENCE
```

8. Reserved Keywords

The following words are reserved in AAL and may not be used as sensor IDs or variable names:

RULE	SEQUENCE	STEP	IF	AND
OR	NOT	FOR	THEN	RESET
WHEN	ON	ENTER	EXIT	WHILE
ACTIVE	ADVANCE	GOTO	TIMEOUT	MODE
SINGLE	CYCLIC	HOLD	PRIORITY	ENABLED
CRITICAL	HIGH	NORMAL	LOW	RELAY
PID	SMS	EMAIL	LOG	SET
EXPR	COUNTER	VARIABLE	TRIGGER	START
STOP	ALWAYS	RISING	FALLING	NO_DATA
TIME	BETWEEN	AFTER	BEFORE	DAY
IN	IS	TRUE	FALSE	END

9. Standard Math Library

The following functions are available in EXPR conditions and SET EXPR actions:

Function	Signature	Description
abs	abs(x)	Absolute value
round	round(x, n)	Round x to n decimal places
min	min(a, b, ...)	Minimum of arguments
max	max(a, b, ...)	Maximum of arguments
sqrt	sqrt(x)	Square root
floor	floor(x)	Round down to integer
ceil	ceil(x)	Round up to integer
pow	pow(x, y)	x raised to power y
log	log(x)	Natural logarithm
log10	log10(x)	Base-10 logarithm

Function	Signature	Description
sin	sin(x)	Sine (x in radians)
cos	cos(x)	Cosine (x in radians)
pi	pi	Constant $\pi = 3.14159...$
e	e	Constant $e = 2.71828...$

10. Runtime Behaviour

10.1 Conflict Resolution

When multiple rules attempt to control the same relay channel, the following precedence applies:

15. CRITICAL priority rule takes absolute control. No other rule can override it.
16. HIGH priority rule takes control when no CRITICAL rule is active for the same relay.
17. PID controller maintains control when no HIGH or CRITICAL rule is active.
18. NORMAL and LOW priority rules can control relays not claimed by higher-priority rules or PID.

10.2 State Persistence

The following state is persisted to the SQLite database and survives system restart:

- Sequence current step and run count
- Virtual variable values
- Event counter values and timestamps
- Rule trigger history (for cooldown enforcement)
- Retentive timer accumulated values

10.3 Error Handling

If a rule evaluation encounters an error (sensor not found, expression parse error, hardware fault), the runtime:

19. Logs the error to the system log with timestamp and rule name
 20. Skips the current evaluation cycle for that rule
 21. Does NOT modify any relay state
 22. Continues evaluating all other rules
 23. Notifies the operator via email if error persists for more than 3 consecutive cycles
-

11. Certification and Intellectual Property

11.1 Copyright

The AKILI Automation Language (AAL), including this specification document, all associated syntax definitions, runtime implementation, and standard library, is the exclusive intellectual property of CrossTech Path, registered in France.

© 2026 CrossTech Path. All rights reserved.

Unauthorized reproduction, modification, or distribution of this specification or any part thereof is prohibited without express written permission from CrossTech Path.

11.2 Trademark

"AKILI", "AAL", "AKILI Automation Language", and the AKILI logo are trademarks of CrossTech Path. Use of these marks requires written authorization.

11.3 Prior Art Declaration

CrossTech Path hereby declares that the AKILI Automation Language constitutes an original work created by Eric Fanane during the period January–May 2026 in France. This document establishes the date of first publication of the AAL specification for the purpose of establishing prior art and copyright ownership.

The following elements are claimed as original intellectual property:

- The AKILI Automation Language syntax as defined in this document
- The natural-language-style rule structure combining IF/AND/OR/THEN/RESET WHEN constructs
- The integration of SFC-style sequences with event-driven rules in a single DSL
- The combination of PID control override through a natural language rule system
- The AKILI Rule Engine runtime architecture as implemented in the AKILI IoT Platform

11.4 Open Standard Intent

CrossTech Path intends to submit AAL v1.0 to a recognized standards body for consideration as an open standard for industrial automation in developing regions. Until such time as a formal standard is adopted, this document represents the authoritative specification.

11.5 Third-Party Implementations

Third parties wishing to implement an AAL-compatible runtime or tool must obtain a license from CrossTech Path and submit to a conformance test suite to be published by CrossTech Path.

12. Revision History

Version	Date	Author	Changes
1.0.0	May 2026	E. Fanane	Initial release — AAL v1.0 complete

Version	Date	Author	Changes
			specification

Document Approval

Role	Name	Signature / Date
Author	Eric Fanane	____Ericfanane_____
Technical Review	_____	_____
Legal Review	_____	_____
Approved By	_____	_____

— END OF DOCUMENT —